

Backend Interview Notes — Simple Language

1. Client-Server Basics

- **Backend** = server-side code, handles data/logic/security. Client (browser/app) never sees it directly.
- **Request-Response loop**: client sends request → server processes → server sends response. Everything in backend fits this loop.
- **HTTP** = the rulebook for how requests/responses are structured.
 - Methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
 - Status codes: 2xx = success, 3xx = redirect, 4xx = client's mistake, 5xx = server's mistake
- **REST** = design convention — organize routes around resources (nouns), use HTTP methods for actions.

```
GET    /tasks      → list all
GET    /tasks/:id  → get one
POST   /tasks      → create
PATCH /tasks/:id  → partial update
DELETE /tasks/:id  → delete
```

Interview line: "REST organizes APIs around resources, using HTTP methods to represent actions on them."

2. TypeScript — Why and Core Ideas

- **Why TS over plain JS**: catches type errors while coding, not after deploy. Big codebases stay safer.
- `strict: true` — turns on all strict checks. Most important ones:
 - `strictNullChecks` — forces you to handle `null`/`undefined` explicitly, prevents "cannot read property of undefined" crashes
 - `noImplicitAny` — errors if TS can't figure out a type, instead of silently allowing anything
- **Never turn `strict` off** — without it, TS gives you zero real safety, just looks like TS.

Interview line: "Strict mode is what makes TypeScript actually catch bugs at compile time instead of runtime — turning it off defeats the purpose."

3. Layered Architecture (Controller → Service → Repository)

- **Controller** — reads request, calls service, sends response. No business logic, no DB code.
- **Service** — business logic/rules live here (e.g. "title is required"). Calls repository. Never touches `req`/`res`.

- **Repository** — only place that touches the actual data (array or DB). No business logic.

Flow: `route → controller → service → repository → data`

Why it matters: swap the database later (array → Postgres) — only repository changes, rest of app untouched. Also makes services testable without a real DB or real HTTP request.

Interview line: "Controller is the traffic cop, service is the brain, repository is the DB hands. Each layer only talks to the one directly below it."

4. Environment Config (Zod + dotenv)

- **Why validate env vars:** `process.env.PORT` is always a string (or `undefined`) — trusting it blindly risks silent bugs (e.g. wrong port, typo'd `NODE_ENV`).
- **Fail-fast principle:** if config is invalid, crash immediately at startup with a clear error — don't let the app run with broken config and fail mysteriously later.
- **Zod pattern:**

```
ts

const envSchema = z.object({
  PORT: z.coerce.number().int().positive().default(4000),
  NODE_ENV: z.enum(['development', 'production', 'test']).default('development')
});

const parsed = envSchema.safeParse(process.env);
if (!parsed.success) { throw new Error('Invalid env'); }
export const env = parsed.data;
```

- `z.coerce.number()` — converts string → number before checking (env vars are always strings).
- `safeParse` — returns `{success, data}` or `{success, error}`, doesn't throw automatically (you control what happens).

Interview line: "I validate environment variables at boot using Zod so the app fails fast with a clear error instead of crashing unpredictably mid-request."

5. Centralized Error Handling

- **Two kinds of errors:**
 - **Operational (expected)** — e.g. "task not found," bad input. Safe to show clean message to client.
 - **Programmer (unexpected)** — real bugs. Log full detail, never leak internals (security risk — stack traces reveal file paths).
- **Custom error class:**

```
ts
```

```
export class HttpError extends Error {
  statusCode: number;
  constructor(message: string, statusCode: number) {
    super(message);
    this.name = 'HttpError';
    this.statusCode = statusCode;
  }
}
```

- **Express error middleware** — special signature: **4 params**, **err** first. Express auto-detects this shape.

ts

```
app.use((err, req, res, next) => {
  // handle and format error response here
});
```

Must be registered **last**, after all routes.

- **catchAsync** pattern (Express 4) — wraps async handlers so thrown errors reach the error middleware. (Express 5, which you're using, does this automatically.)

Interview line: "I use one centralized error-handling middleware so every error in the app — thrown or passed to next() — gets consistent logging and response formatting in exactly one place."

6. Zod Validation (Input Validation)

- **Where it belongs:** controller layer — validate at the boundary before business logic runs.
- **Why not manual if checks:** doesn't scale (type checks, length checks, multiple fields get messy), Zod does it declaratively.
- **Pattern:**

ts

```
const schema = z.object({
  title: z.string().min(1, 'Title is required'),
});
const parsed = schema.parse(req.body); // throws ZodError if invalid
```

- **.parse()** throws automatically on failure (Express 5 auto-catches thrown errors in routes).
- **.optional()** — field may be missing, used for PATCH (partial updates).

- `z.infer<typeof schema>` — derive a TypeScript type directly from your Zod schema. One source of truth for both validation rules AND types — no risk of them drifting apart.

Interview line: "I validate all untrusted input — env vars, request bodies — at the boundary using Zod, and derive TypeScript types directly from the same schema so validation and types can't drift apart."

7. Structured Logging (Winston)

- **Why not** `console.log`: no log levels, no structured/searchable output, no environment-specific behavior.
- **Log levels** (most → least severe): `error > warn > info > http > verbose > debug > silly`
 - Setting `level: 'info'` shows `error/warn/info`, hides `debug` (noise).
 - Dev: `level: 'debug'` (see everything). Prod: `level: 'info'` (only important stuff).
- **Format:** combine timestamp + JSON — structured, machine-searchable (important for real production log tools).
- **Transport:** where logs go — console, file, external service.
- Use the log method matching severity: `logger.info()` for normal events, `logger.warn()` for validation failures, `logger.error()` for real errors.

Interview line: "Log levels let me control verbosity per environment without code changes — full detail in dev, only meaningful events in production."

8. TypeScript Config Details (tsconfig.json)

- `rootDir` / `outDir` — where TS source lives vs where compiled JS output goes. Keeps them separate (`dist` gets git-ignored).
 - `noUncheckedIndexedAccess` — array/object access (`arr[5]`) returns `T | undefined`, not just `T` — forces you to handle "might not exist."
 - `exactOptionalPropertyTypes` — distinguishes "property missing entirely" vs "property present but explicitly `undefined`." Can cause type mismatches when mixing `Partial<T>` with Zod's `.optional()` output — fix by using `z.infer<typeof schema>` instead of hand-written `Partial<T>` types.
 - `verbatimModuleSyntax` — forces explicit `import type` for type-only imports (erased at compile time, zero runtime cost).
 - `module: "nodenext"` vs `moduleResolution: "bundler"` — `nodenext` mirrors raw Node's strict ESM rules (needs `.js` extensions in imports even in `.ts` files). `bundler` mode allows extensionless imports (works when using a runner like `tsx`).
-

9. Module System (import/export)

- **CommonJS (old):** `require()` / `module.exports`
- **ES Modules (modern, what we use):** `import` / `export`
- `package.json`'s `"type": "module"` tells Node to treat files as ESM.
- **Import types:**

ts

```
import express from 'express'; // default import
import { Router } from 'express'; // named import
import type { Request } from 'express'; // type-only import (erased at compile time)
```

10. TypeScript Utility Types

- `Partial<T>` — makes every property of `T` optional. Used for PATCH-style partial updates.
- `z.infer<typeof schema>` — derives a plain TS type from a Zod schema (preferred over `Partial<T>` when the shape must match Zod's validated output exactly).

11. Common JS/TS Gotchas Covered

- `==` vs `===` — always use `===` / `!==`. `==` does type coercion (`"5" == 5` is `true`) — unpredictable, considered a mistake in modern code.
- `.find()` vs `.findIndex()` — `.find()` returns the matching element (or `undefined`). `.findIndex()` returns its position (needed for `.splice()` to remove/replace).
- **Spread operator** `{...obj}` — copies properties into a new object. Used for merging updates: `{ ...oldTask, ...updates }`.
- `instanceof` — checks if an error/object was created from a specific class. Used to branch logic: `if (err instanceof ZodError) {...}`.
- **Type narrowing** — after a guard check (`if (!id) throw ...`), TypeScript "knows" the remaining code path has a safe, non-null type.

12. Tooling (Prettier, Husky)

- **Prettier** — formatting only (spacing, quotes, semicolons). Not a linter, doesn't check logic/bugs.
- **ESLint** (separate tool) — logic/bug rules, different job from Prettier.
- **Husky** — manages git hooks (scripts that run at git events like `pre-commit`).
- **lint-staged** — runs formatting/linting only on staged files (fast, not the whole project).
- **Flow:** `git commit` → Husky's `pre-commit` hook → `lint-staged` → Prettier/ESLint run on staged files → blocks commit if it fails.

13. Testing (Jest + Supertest)

- **Why write tests:** proves code still works after changes, without manually re-testing every endpoint by hand each time.
- **Unit test** — tests one function alone. **Integration test** — tests multiple pieces together (e.g., full request → response cycle).
- **Core syntax:**

ts

```
describe('group name', () => {  
  it('specific test case', async () => {  
    expect(actual).toBe(expected);  
  });  
});
```

- `describe` — groups tests (organization only, no logic)
- `it` — one actual test
- `expect(x).toBe(y)` — assertion, fails the test if not equal
- **Supertest** — simulates HTTP requests directly against your Express `app` object, no real server/port needed:

ts

```
const res = await request(app).get('/api/v1/tasks');  
const res2 = await request(app).post('/api/v1/tasks').send({ title: 'x' });
```

- **Common Windows/ESM setup gotchas** (real issues hit, worth remembering exist, not memorizing fixes):
 - Jest's `.bin/jest` shell script fails on Windows — call `node_modules/jest/bin/jest.js` directly instead
 - Test files must match Jest's naming pattern: `*.test.ts` (not `.tests.js` or similar)
 - TS needs `"jest"` added to `tsconfig.json`'s `"types"` array so `describe`/`expect`/`it` are recognized
 - `esModuleInterop: true` needed in `tsconfig.json` for older CommonJS packages (like `dotenv`) to import correctly under ts-jest

Interview line: "I write integration tests with Jest and Supertest that hit the Express app directly in-memory, covering both success and validation-failure paths for each endpoint."

15. PostgreSQL + Prisma (ORM)

- **Why a real database:** in-memory arrays vanish on restart. Real apps need data to persist and survive crashes/restarts/multiple server instances.
- **Why PostgreSQL (relational):** data has real relationships (Task belongs to User, etc). Relational DBs enforce structure + integrity via foreign keys.
- **Core DB terms, plain:**
 - **Table** — like a spreadsheet: rows = records, columns = fields
 - **Row** — one record
 - **Primary key** — uniquely identifies each row (`(id)`), no duplicates allowed
 - **Foreign key** — a column pointing to another table's primary key — this is how relationships are enforced
 - **Schema** — overall structure: tables, columns, types, relationships
- **ORM (Object-Relational Mapper) — what Prisma is:** lets you write `prisma.task.findMany()` in TypeScript instead of raw SQL strings. Fully typed, catches typos/mistakes at compile time, avoids manual SQL injection risk.
- **Prisma workflow:**
 1. Define models in `prisma/schema.prisma`
 2. Run `prisma migrate dev --name <description>` — compares schema to actual DB, generates a migration file (versioned SQL), applies it, regenerates the typed client
 3. Use the generated `PrismaClient` in code — fully typed based on your schema
- **Example model:**

```
prisma

model Task {
  id      String   @id @default(cuid())
  title   String
  completed Boolean @default(false)
  createdAt DateTime @default(now())
}
```

- `@id` — marks primary key
- `@default(cuid())` — auto-generates a unique collision-resistant ID
- `@default(false)` / `@default(now())` — default values if not provided
- **Migrations = version-controlled schema changes.** Never manually edit the DB structure by hand in a real team — breaks reproducibility across devs/deployments. Every schema change goes through a migration file.
- **Real-world gotcha hit:** newest Prisma major version (7) moved DB connection config out of `schema.prisma` into a separate `prisma.config.ts` + adapter pattern — a big breaking change, poorly covered in tutorials/docs still. Downgraded to Prisma 6

(stable, matches nearly all existing documentation) — same reasoning as the earlier TypeScript 7 downgrade. Lesson: **cutting-edge major versions often break ecosystem compatibility** — check what's actually documented/stable before pinning latest.

- **Windows PATH gotcha:** `npx prisma <command>` can silently resolve to a different/latest cached version instead of your project's local install. Fix: call the local binary directly — `node_modules\.bin\prisma <command>`.
- **Async ripples upward through layers.** Once repository functions became `async` (talking to a real DB takes time), every function that calls them — service, then controller — also had to become `async` and use `await`. This is a very common real pattern: introducing any async operation forces every caller above it in the chain to become async too.
- **Different Prisma methods fail differently — match your error handling to that:**
 - `findUnique`, `findMany` — return `null` / empty array when nothing found. Use a plain `if (!result)` check.
 - `update`, `delete` — **throw** an error if the record doesn't exist. Use `try/catch` to convert that into your own clean `HttpError(404)`.
 - Don't blanket-wrap everything in try/catch "just in case" — it hides which methods actually need it and can accidentally mislabel real errors (like a lost DB connection) as 404s.
- `exactOptionalPropertyTypes` **friction lesson:** this strict-but-optional TS flag (distinguishes "key missing" vs "key explicitly undefined") kept causing type mismatches every time Zod's `.optional()` output met a differently-shaped type (`Partial<T>`, then Prisma's own generated types). Turned it off — it's not part of core `strict: true`, rarely matters in real code, and was purely colliding with two libraries that don't share its convention. Core safety (`strict`, `noUncheckedIndexedAccess`, `strictNullChecks`) stayed on.

Interview line: "Introducing a database made every layer async — I had to propagate `async`/`await` from repository up through service and controller, and match error handling to how each Prisma method actually fails: null-checks for lookups, try/catch for update/delete."

17. Test Database Isolation

- **Why tests need a separate database:** once your app uses a real DB, tests running against it would leave junk data behind forever, and tests could interfere with each other's data.
- **Pattern:** separate test database (e.g. `_test_db` suffix), separate `.env.test` file pointing to it, test script loads that file instead of the normal `.env`.
- **Test isolation principle:** every test should start from a clean, known state — never depend on leftover data from a previous test run.
- **Jest lifecycle hooks:**


```
ts

afterEach(async () => {
  await prisma.task.deleteMany(); // wipe data after every test
});

afterAll(async () => {
  await prisma.$disconnect(); // close DB connection once all tests finish
});
```

- `afterEach` — runs after every single test
- `afterAll` — runs once, after the whole suite finishes
- Never run `deleteMany()` with no filter against a real/dev database — only safe on a dedicated test DB.
- **Chaining requests in tests** — when a test needs a real, existing id (for PATCH/DELETE), first create a record via POST, then read its id from the response: `created.body.id`. Can't hardcode ids since the DB generates random ones.

```
ts

const created = await request(app).post('/api/v1/tasks').send({ title: 'x' })
const res = await request(app).patch(`/api/v1/tasks/${created.body.id}`).send()
```

Interview line: "Tests run against a dedicated test database, isolated from dev/prod data, with cleanup hooks that reset state after every test so results stay independent and repeatable."

19. Authentication — Password Hashing (bcrypt)

- **Why passwords are never stored as plain text:** if the database ever leaks, plain text passwords expose every user immediately — and people reuse passwords across sites, making it worse.
- **Hashing = one-way transformation.** `"mypassword"` → `"2b$10 ..."`. Can't reverse it back to the original. Login verification works by hashing the *attempted* password the same way and comparing hashes — never comparing/storing the real password.
- **bcrypt** — industry-standard hashing library for passwords. Auto-adds a **salt** (random data mixed in) so identical passwords don't produce identical hashes — defeats precomputed "rainbow table" attacks.
- **Core functions:**

```
ts
```

```
const hashed = await bcrypt.hash(plainPassword, 10); // 10 = salt rounds
const isMatch = await bcrypt.compare(plainPassword, hashed); // true/false
```

Both async — hashing is intentionally slow (security feature, resists brute-forcing).

- **Never return password/hash fields in API responses** — even hashed. Explicitly pick safe fields to return (`{ id, email }`), never forward the raw DB object.
- **409 Conflict status code** — used when a request conflicts with existing data (e.g. duplicate email on signup). Different from **400** (malformed request) — the request itself is well-formed, it just collides with existing state.
- **Zod v4 note:** string-format validators like `.email()` moved to top-level functions (`z.email()` instead of `z.string().email()`) — old chained form still works but is deprecated.

Interview line: "Passwords are hashed with bcrypt before storage, using a salt to prevent rainbow-table attacks — the app never stores or returns plain-text or hashed passwords in any response."

21. Authentication — JWT (Access + Refresh Tokens)

- **What a JWT is:** a signed piece of text the server gives the client after login. Client sends it back on future requests to prove identity, without re-sending email/password each time.
- **Structure:** `header.payload.signature` — payload is **readable but not editable**. Anyone can decode and read it (not encrypted, just encoded), but only the server (holding the secret) can produce a valid signature for it. Never put sensitive data in the payload.
- **Why two tokens:**
 - **Access token** — short-lived (e.g. 15 min), sent on every request, proves identity.
 - **Refresh token** — long-lived (e.g. 7 days), used only to get a new access token without forcing re-login.
 - Splitting limits damage if a token is stolen — a stolen access token expires fast; the refresh token is used less often and can be revoked/stored more carefully.
- **Different secrets for each token type** — if one leaks, the other stays safe.
- **Core syntax:**

ts

```
jwt.sign({ userId }, secret, { expiresIn: '15m' }); // synchronous, not async
```

- **Same error message for both login failure cases** (`'Invalid credentials'`, **401**) whether email doesn't exist or password is wrong — prevents attackers from figuring out which emails are registered by testing different addresses.

- **401 Unauthorized** — missing/wrong credentials. Different from **403 Forbidden** (identified but not allowed to do this specific thing).
- **Recurring lesson — stale processes strike again:** spent significant time debugging "Cannot POST /login" that looked like a routing bug, but the real cause was an old `npm run dev` process (started 40 minutes earlier) still bound to the port, never having the new route. The giveaway: `[nodemon] clean exit` appearing right after "Server running" — a genuinely live server should never let its process exit on its own.
Standing rule now: if a code change doesn't seem to take effect, check `Get-Process -Name node | Select Id,StartTime` and `netstat -ano | findstr :<port>` before doubting the code.

Interview line: "Login issues a short-lived access token and a longer-lived refresh token, signed with separate secrets — this limits the blast radius if a token is compromised, since the access token expires quickly."

20. Debugging Lesson (real-world, worth remembering)

- Faced a mystery where code changes didn't reflect in server responses.
- Root cause: **stale/orphaned Node processes** still running and bound to the port, silently answering requests instead of the current dev server.
- Fix: `netstat -ano | findstr :4000` to find the PID bound to a port, `Get-Process` to cross-check, `Stop-Process -Force` to kill stray processes.
- **Lesson:** always confirm which process is actually serving requests when behavior seems inexplicable — very common real issue, especially on Windows.

Quick Interview Soundbites (memorize these phrasings)

1. "I use a layered architecture — controller, service, repository — so each layer has a single responsibility and the app stays testable and swappable."
2. "Environment variables are validated at boot with Zod, following a fail-fast principle."
3. "All errors flow through one centralized error-handling middleware, so logging and response formatting stay consistent."
4. "Input validation happens at the controller boundary using Zod, and I derive TypeScript types from the same schema to avoid duplication."
5. "Logging uses structured JSON with levels, so verbosity is controlled per environment without code changes."
6. "Strict TypeScript settings like `strictNullChecks` and `noUncheckedIndexedAccess` catch a whole class of runtime bugs at compile time."